Reviewer Pack — Second-Read Commutator Frobenius Excess Bounds Read-Twice BP…

Entry 9b0a625047c3 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-08 04:12:36 UTC

Second-Read Commutator Frobenius Excess Bounds Read-Twice BP Size

Entry ID: `9b0a625047c3` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-08 04:12:36 UTC

1. Conjecture

Field A (mathematical branch): Free probability / non-commutative L^2: Haagerup-style commutator inequalities and the asymptotic-freeness defect between layer transition matrices that touch the same variable twice **Field B** (complexity object): Read-twice branching program size for IP_2 = ⊕_{i=1}^n x_i·y_i (Borodin–Razborov–Smolensky 1993 read-twice barrier)

Statement:

> For a read-twice BP P of size S over variables v_1,...,v_m, let v_i be read at layers t_i < t'_i with 0/1-transition matrices $A_i^{0,A_i}1 \in \{0,1\}^{S\times S}$ (first read) and $B_i^{0,B_i}1$ (second read). Define the variable-symmetrized second-read commutator $C_i := (A_i^{0+A_i}1)(B_i^{0+B_i}1) - (B_i^{0+B_i}1)(A_i^{0+A_i}1)$, and the invariant ρ(P) := log_2(1 + max_i ||C_i||_F^2). Conjecture: there exists a universal constant α ≤ 4 such that (i) every read-twice BP of size S satisfies ρ(P) ≤ α·log_2 S, while (ii) every read-twice BP computing IP_2 satisfies ρ(P) ≥ n/4; together these would force S ≥ 2^{n/(4α)} on IP_2.

Rationale (proposer's reasoning):

> In free probability, two operator families that act on disjoint variable supports become asymptotically free and have small commutator Frobenius norm; reading a variable twice forces non-trivial algebraic interaction whose Hilbert–Schmidt size is, in Cook–Reckhow style, bookkeeping for how much 'fresh' state the BP must allocate to remember the first read. The commutator C_i is exactly the obstruction that any extension of bounded arithmetic capturing read-twice BPs would have to certify, so a polylogarithmic upper bound in S on ρ would translate (via Buss-style witnessing) to a witnessing protocol of size O(log S), giving the IP_2 lower bound by mismatch with Forster-style discrepancy. The bridge is computationally light because C_i is a single 4-term matrix product on 0/1 matrices.

Taxonomy category: `BOUNDED_ARITHMETIC` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `e34d20a3e288bad5`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> Across 480 random read-twice BP trials (30 seeds × 4 sizes S∈{8,16,32,64} × 4 var-counts m∈{8,12,16,20}), every instance must satisfy ρ(P) ≤ 4·log_2 S; and for all 4 IP_2 calibration points n∈{3,4,5,6}, ρ(P) ≥ n/4 must hold. Support requires both upper-bound and IP_2 lower-bound to hold on 100% of instances.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.92	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.82	SAFE	SAFE
ALGEBRIZATION	SAFE	0.70	SAFE	SAFE
KARP_LIPTON	SAFE	0.86	SAFE	SAFE

4. Novelty audit

Verdict: `NOVEL` against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): `- read-twice branching program lower bound inner product commutator - Frobenius norm commutator transition matrix branching program size - free probability asymptotic freeness branching program IP_2 Borodin Razborov Smolensky`

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_bp(S, m):
        layers = [random.sample(range(S), S) for _ in range(m)]
        A = [[0] * S for _ in range(m)]
        B = [[0] * S for _ in range(m)]

        for i in range(m):
            for j in range(S):
                A[i][j] = random.choice([0, 1])
                B[i][j] = random.choice([0, 1])

        return layers, A, B

    def matrix_multiply(A, B):
        S = len(A)
        C = [[0] * S for _ in range(S)]
        for i in range(S):
            for j in range(S):
```

```
            for k in range(S):
                C[i][j] += A[i][k] * B[k][j]
    return C

def frobenius_norm(A):
    S = len(A)
    norm = 0
    for i in range(S):
        for j in range(S):
            norm += A[i][j] ** 2
    return math.sqrt(norm)

def compute_rho(C):
    max_norm = max(frobenius_norm(row) for row in C)
    return math.log2(1 + max_norm ** 2)

def is_ip2(n):
    layers = [[i, i+1] for i in range(n)]
    A = [[0 if j != k else 1 for j in range(2**n)] for k in range(2**n)]
    B = [[0 if j != k else 1 for j in range(2**n)] for k in range(2**n)]
    return layers, A, B

sizes = [8, 16, 32, 64]
var_counts = [8, 12, 16, 20]
alpha = 4
instances_tested = 0
conjecture_holds = True
counterexample = ""

for S in sizes:
    for m in var_counts:
        layers, A, B = generate_bp(S, m)
        C = [matrix_multiply(A[i], B[i]) - matrix_multiply(B[i], A[i]) for i in range(m)]
        rho = compute_rho(C)

        instances_tested += 1
        if rho > alpha * math.log2(S):
            conjecture_holds = False
            counterexample = f"Random BP of size {S} with m={m} variables, ρ(P) = {rho}"

# IP_2 calibration points
ip2_sizes = [3, 4, 5, 6]
for n in ip2_sizes:
    layers, A, B = is_ip2(n)
    C = [matrix_multiply(A[i], B[i]) - matrix_multiply(B[i], A[i]) for i in range(n)]
    rho = compute_rho(C)

    instances_tested += 1
    if rho < n / 4:
        conjecture_holds = False
        counterexample = f"IP_2 BP of size {2**n}, ρ(P) = {rho}"

return {
    "metric_name": "ρ(P)",
    "metric_value": compute_rho(C),
    "instances_tested": instances_tested,
```

```
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(5, 8)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_rho = sum(r["metric_value"] for r in results) / len(results)
    std_rho = math.sqrt(sum((r["metric_value"] - mean_rho) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_rho} std={std_rho} support_fraction={support_fractior
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(s for s, r in zip(seeds, results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{results[0]['counterexample']}\" first_failing_
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_00546418.py", line 104, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_00546418.py", line 70, in run_trial
    C = [matrix_multiply(A[i], B[i]) - matrix_multiply(B[i], A[i]) for i in range(m)]
         ^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_00546418.py", line 39, in matrix_multip
    C[i][j] += A[i][k] * B[k][j]
               ~~~~^^^
TypeError: 'int' object is not subscriptable
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test crashed with a TypeError in matrix_multiply (indexing an int as a 2D matrix), so no data was produced and neither the upper-bound nor the IP_2 lower-bound criteria could be evaluated. | next: Fix matrix_multiply to correctly handle the 2D list-of-lists transition matrices (verify A[i] and B[i] are matrices, not flattened ints) and rerun the 480 random trials plus the 4 IP_2 calibration points.

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	107660
2	preregistration	claude_max	opus	0	0	7420
3	novelty	claude_max	opus	0	0	4690
4	novelty	claude_max	opus	0	0	6465
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	119377
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11951
7	judge	claude_max	opus	0	0	4934

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 262497 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 2}

(full prompt+response transcripts available in `research/audit/9b0a625047c3.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/9b0a625047c3.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/9b0a625047c3.tar.gz` (if generated)